

SIMATS ENGINEERING

Artificial Intelligence (CSA17) — Assessment Tool 1

SOLUTION DESIGN DOCUMENT

Design of Distributed AI Systems using PySpark RDD Operations
and Intelligent Agent Architectures

Course Outcome	CO5 (BTL 5 — Evaluate / Create)
Scope	All 5 Design Questions (Annexure A)
Core Technologies	Apache PySpark, RDDs, Multi-Agent Systems
Document Type	Solution / Design Blueprint

Contents

- 1 1. Hybrid AI Recommendation Engine Design
- 2 2. Smart Disaster Detection System
- 3 3. AI-Based Fraud Detection Framework
- 4 4. Autonomous Supply Chain Optimization System
- 5 5. Personalized Learning Analytics Platform

1. Hybrid AI Recommendation Engine Design

1.1 Problem Understanding

The system must merge two complementary recommendation signals — content similarity (what an item is about) and collaborative signal (what similar users liked) — while scaling to large, continuously growing interaction logs, and must adapt in real time as user behaviour changes.

1.2 System Architecture

- **Ingestion Layer:** raw clickstream/rating events land in Kafka/HDFS and are loaded into a PySpark RDD via `sc.textFile()` or a streaming `DStream`.
- **Distributed Processing Layer:** RDD transformations (`map`, `groupByKey`, `reduceByKey`) compute per-item average ratings (collaborative signal) and per-user genre/feature preference (content signal) in parallel across the cluster.
- **Hybrid Scoring Agent:** an intelligent agent fuses both signals using a weighted score $\text{hybrid} = \alpha \cdot \text{content} + (1 - \alpha) \cdot \text{collaborative}$.
- **Adaptive Feedback Agent:** perceives click/like/skip feedback and updates α in real time (reinforcement-style adaptation), so the engine leans more on whichever signal is currently earning engagement.
- **Serving Layer:** top-N ranked items are cached and served with low latency.

1.3 Role of RDDs / Distributed Processing

RDDs partition the interaction log across worker nodes so that aggregation (average rating per item, per-user history) happens in parallel; RDD lineage also gives automatic fault tolerance — if a worker node fails mid-job, Spark recomputes only the lost partitions instead of restarting the entire computation. This is essential once the interaction log grows into millions/billions of events.

1.4 Role of Intelligent Agents

The Hybrid Scoring Agent and Adaptive Feedback Agent form a perceive-reason-act loop: they perceive new feedback events, reason about which filtering strategy is performing better for a segment of users, and act by re-weighting and re-ranking recommendations — without a human retraining the whole model.

1.5 Scalability & Fault Tolerance

Horizontal scaling is achieved by adding more Spark executors as interaction volume grows; RDD checkpointing periodically persists intermediate state to stable storage to bound recovery time for long-running streaming jobs.

Fig. 1 — Hybrid AI Recommendation Engine Architecture

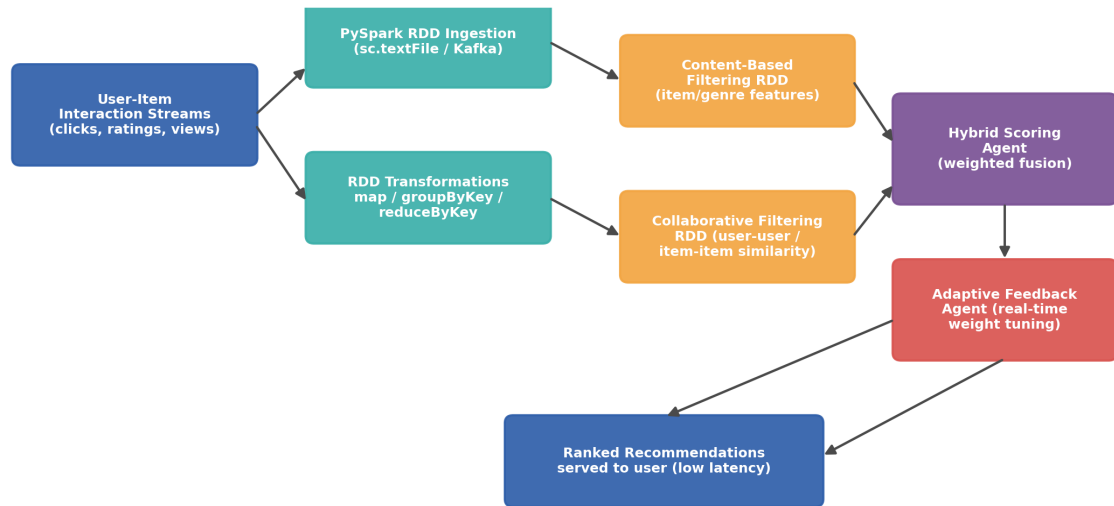


Figure 1: Hybrid recommendation architecture — RDD-based content & collaborative pipelines fused by a scoring agent, tuned by an adaptive feedback agent.

Representative PySpark RDD Snippet:

```
interactions_rdd = sc.parallelize(raw_interactions)
item_avg_rdd = (interactions_rdd
    .map(lambda x: (x[1], x[2]))          # (item, rating)
    .groupByKey()
    .mapValues(lambda r: sum(r) / len(r))) # collaborative signal

class HybridScoringAgent:
    def recommend(self, user, alpha=0.6):
        # content_score + collaborative_score fused with alpha
        ...
```

2. Smart Disaster Detection System

2.1 Problem Understanding

Natural disasters must be detected early by fusing heterogeneous, high-velocity streams — satellite imagery, ground sensors, and noisy social-media text — despite differing formats, sampling rates, and reliability.

2.2 System Architecture

- **Multi-Source Ingestion:** three parallel Spark Streaming pipelines ingest satellite-derived flood masks, sensor telemetry (water level, seismic amplitude), and geotagged social posts.
- **Data Fusion RDD Layer:** each stream is converted to a common (`region`, `signal_value`) RDD; `reduceByKey` aggregates peak readings per region, and a `join/union` combines all three sources into one fused view.
- **Per-Source Detection Agents:** a Satellite Agent, Sensor Agent, and Social-Media Agent each apply a domain-specific threshold/model to their own signal.
- **Coordinator Agent:** performs majority-vote (or Bayesian) fusion across the three agents' votes to decide whether to raise a regional early warning, reducing false positives from any single noisy source.
- **Response Layer:** confirmed alerts trigger dispatch of warnings and a response plan to civil-authority systems.

2.3 Role of RDDs / Distributed Processing

RDD partitioning by region allows all regions to be scored in parallel; `reduceByKey/groupByKey` aggregate bursty sensor and social data efficiently, and Spark's in-memory RDD caching keeps rolling window computations (e.g., last 10-minute peak) fast enough for near-real-time alerting.

2.4 Role of Intelligent Agents

Agents act as specialized, autonomous evaluators of each data modality; the Coordinator Agent performs multi-agent consensus so that no single unreliable source (e.g., social media hype) can trigger a false alarm alone, while a strong corroborated signal still triggers a fast warning.

2.5 Scalability & Fault Tolerance

New regions or sensor types are added simply by adding another RDD source that maps into the same (`region`, `value`) schema; RDD lineage-based recovery keeps the pipeline running even if a subset of streaming workers fail during a disaster event — precisely when reliability matters most.

Fig. 2 — Smart Disaster Detection System Architecture

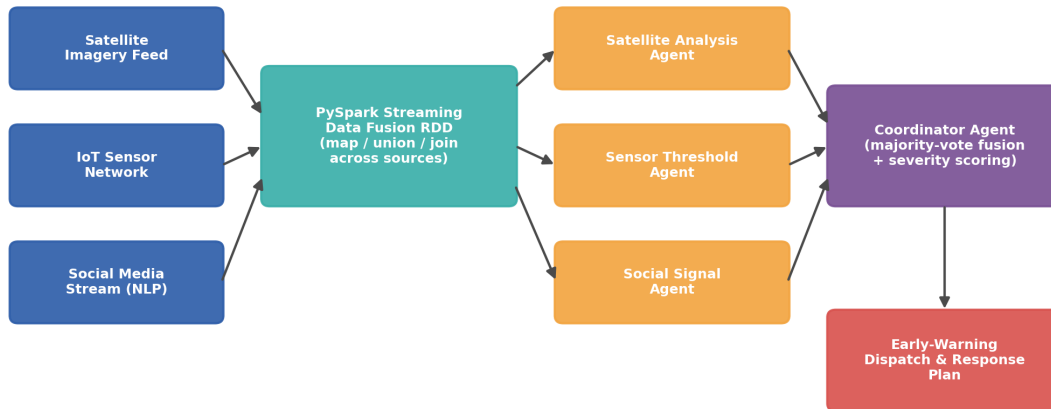


Figure 2: Multi-source disaster detection pipeline with per-source agents and a majority-vote coordinator agent.

Representative PySpark RDD Snippet:

```
sensor_rdd = sc.parallelize(sensor_stream).reduceByKey(max)
sat_rdd     = sc.parallelize(satellite_stream).reduceByKey(max)
social_rdd  = sc.parallelize(social_stream).reduceByKey(lambda a, b: a + b)

class CoordinatorAgent:
    def fuse(self, votes):
        return sum(votes) >= 2  # majority vote across agents
```

3. AI-Based Fraud Detection Framework

3.1 Problem Understanding

Financial fraud detection must process a very high volume of transactions with low latency, catch statistically unusual behaviour per account, and keep improving as fraud patterns evolve — all without blocking legitimate transactions.

3.2 System Architecture

- **Ingestion & Partitioning:** the transaction stream is parallelized into an RDD and partitioned by account ID so each partition can be profiled independently.
- **Statistics RDD:** `groupByKey` + `mapValues` compute a rolling mean and standard deviation of transaction amount per account.
- **Anomaly Detection Agent:** scores every new transaction with a z-score $((\text{amount} - \text{mean}) / \text{std})$ or a distributed ML classifier (e.g., Spark MLlib Random Forest), flagging outliers beyond a threshold.
- **Alert & Case-Management Agent:** receives flags, opens investigation tickets, and routes high-severity cases to analysts.
- **Feedback / Continuous-Learning Loop:** analyst confirmations (true/false positive) feed back into a periodic RDD-based batch retraining job that refines thresholds/model weights.

3.3 Role of RDDs / Distributed Processing

Partitioning by account lets thousands of per-account profiles be computed concurrently across the cluster; RDD transformations such as `filter` then isolate only the anomalous subset for downstream alerting, avoiding a full-dataset scan on every check.

3.4 Role of Intelligent Agents

The Anomaly Detection Agent autonomously scores and flags transactions; the Alert Agent manages the human-in-the-loop response; together they close a detect → alert → learn loop that keeps improving precision/recall over time.

3.5 Scalability & Fault Tolerance

Horizontal partitioning scales linearly with transaction volume; RDD lineage guarantees no transaction statistics are silently lost on node failure, which is a compliance requirement in financial systems.

Fig. 3 — AI-Based Fraud Detection Framework Architecture

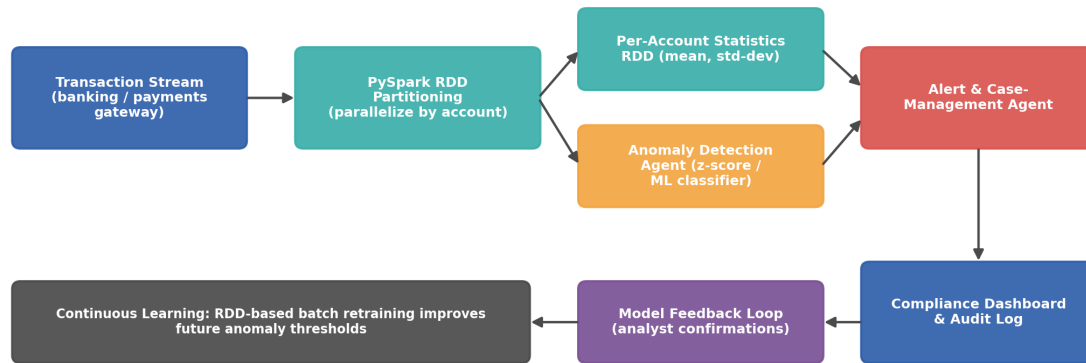


Figure 3: Distributed anomaly-scoring pipeline with an Anomaly Detection Agent and Alert/Case-Management Agent, closed by a continuous-learning feedback loop.

Representative PySpark RDD Snippet:

```
stats_rdd = (txn_rdd.map(lambda x: (x[0], x[1]))
              .groupByKey()
              .mapValues(lambda amts: {'mean': mean(amts), 'std': std(amts)}))

class AnomalyAgent:
    def score(self, acc, amount):
        z = (amount - stats[acc]['mean']) / stats[acc]['std']
        return z, abs(z) >= self.z_threshold
```


4. Autonomous Supply Chain Optimization System

4.1 Problem Understanding

Supply-chain optimization must forecast demand per warehouse and product, then decide inventory transfers/reorders across a network of warehouses in a way that minimizes cost, without a single point of centralized control becoming a bottleneck.

4.2 System Architecture

- **Data Layer:** point-of-sale, inventory, and logistics IoT feeds are unified into an RDD of (warehouse, product, demand) records.
- **Demand Forecasting RDD Pipeline:** `groupByKey` + `mapValues` compute a moving-average forecast (with a safety margin) per warehouse-product pair.
- **Warehouse Agents (decentralized):** one autonomous agent per warehouse compares its own forecast against current stock and computes its surplus/deficit — decisions are made locally, not by a single central optimizer.
- **Logistics Coordinator Agent:** nets out surpluses against deficits across warehouses, proposing inter-warehouse transfers first (cheaper than new orders) and falling back to new orders only for the unmet remainder.

4.3 Role of RDDs / Distributed Processing

RDD transformations let demand forecasting run in parallel across every warehouse-product combination; this scales to large retail networks with thousands of SKUs without redesigning the pipeline.

4.4 Role of Intelligent Agents

This is a genuinely **multi-agent** design: each Warehouse Agent is autonomous and self-interested (minimize its own stockout risk), while the Coordinator Agent performs light-touch arbitration — a decentralized-decision, centrally-arbitrated pattern that mirrors real supply-chain organizations and avoids a single point of failure.

4.5 Scalability & Fault Tolerance

Because each warehouse's evaluation is independent, new warehouses/products are added without touching existing agents; if the Coordinator Agent is temporarily unavailable, Warehouse Agents can still fall back to local reorder rules, ensuring graceful degradation.

Fig. 4 — Autonomous Supply Chain Optimization Architecture

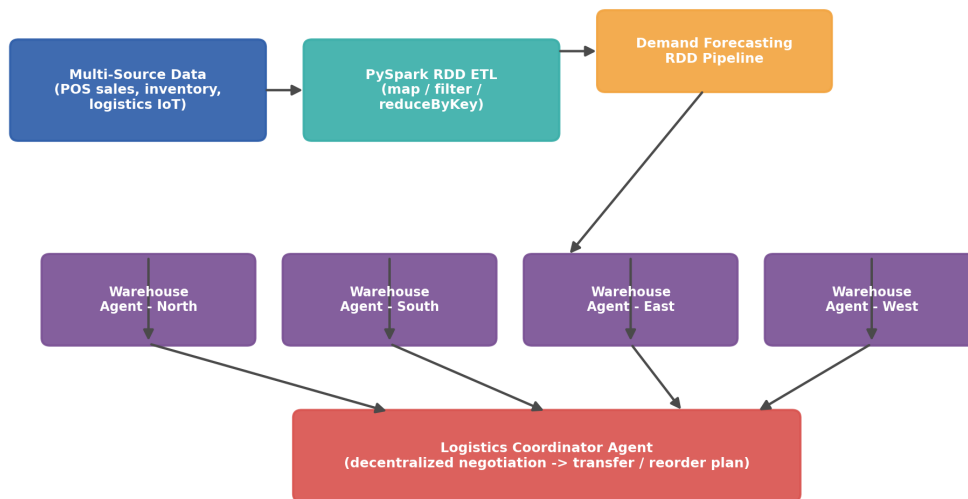


Figure 4: Decentralized Warehouse Agents feeding a Logistics Coordinator Agent that produces the final transfer/reorder plan.

Representative PySpark RDD Snippet:

```
forecast_rdd = demand_rdd.groupByKey().mapValues(
    lambda d: sum(d)/len(d) * 1.1) # +10% safety margin

class LogisticsCoordinatorAgent:
    def optimize(self, evaluations):
        # net surplus against deficit, propose transfers, then new orders
        ...
```

5. Personalized Learning Analytics Platform

5.1 Problem Understanding

The platform must analyse learning-behaviour data (quiz scores, time-on-task, topic attempts) for thousands of learners and personalize what each learner sees next, in real time, at scale.

5.2 System Architecture

- **Ingestion:** LMS clickstream and quiz-attempt logs are parallelized into an RDD of (student, topic, score, time) records.
- **Mastery Scoring (RDD Aggregation):** groupByKey on (student, topic) computes a mastery score per topic per learner.
- **Adaptive Tutor Agent (per learner):** perceives a learner's mastery profile and selects the next best topic and difficulty level — an autonomous, personalized recommendation loop.
- **At-Risk Detection Agent:** scans class-wide mastery to flag learners falling below a risk threshold for instructor intervention.
- **Instructor Dashboard:** aggregated, RDD-computed class analytics (heatmaps, risk reports) support human oversight of the automated tutoring loop.

5.3 Role of RDDs / Distributed Processing

RDD aggregation makes it feasible to recompute mastery scores for an entire cohort (thousands of learners × many topics) every time new interaction data streams in, which would be far too slow with a single-machine batch job.

5.4 Role of Intelligent Agents

The Adaptive Tutor Agent closes a perceive (mastery data) → decide (next topic/difficulty) → act (serve content) → observe (new attempt) loop per learner; the At-Risk Agent adds a safety layer that surfaces cases needing a human instructor rather than fully automating intervention.

5.5 Scalability & Fault Tolerance

Because mastery computation and next-topic selection are both keyed by student ID, the workload partitions naturally across the cluster; adding more learners or courses scales horizontally without architectural change.

Fig. 5 — Personalized Learning Analytics Platform Architecture

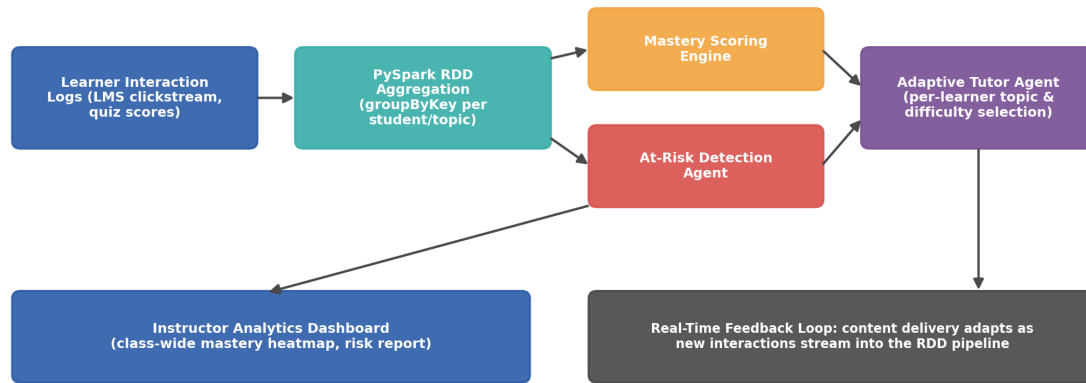


Figure 5: RDD-based mastery scoring feeding a per-learner Adaptive Tutor Agent and a class-wide At-Risk Detection Agent.

Representative PySpark RDD Snippet:

```
mastery_rdd = (logs_rdd.map(lambda x: ((x[0], x[1]), x[2]))
    .groupByKey()
    .mapValues(lambda s: sum(s)/len(s)))

class AdaptiveTutorAgent:
    def next_topic(self, student):
        weakest = min(topics, key=lambda t: mastery[student].get(t, 0))
        return weakest
```

Closing Note

All five designs follow a consistent pattern: (1) an RDD-based distributed data-processing layer that guarantees horizontal scalability and fault tolerance, and (2) a layer of intelligent, autonomous agents that perceive processed data, reason about it, and act — individually or through coordination/consensus — to fulfil each system's real-time objective. A working reference implementation of every pipeline (with synthetic data) is provided in the accompanying Python file and its console/JSON output.